EXPERIMENT 10

Shravanya Andhale
D15A-30

AIM: Explore and implement Autoencoders for image denoising.

THEORY:

1. Dataset Source

Dataset: MNIST Handwritten Digit Dataset

Source: Keras built-in dataset

Contains 70,000 grayscale images (60,000 training and 10,000 testing)

2. Dataset Description

Image size: 28 × 28 pixels

Grayscale images

Pixel values normalized between 0 and 1

Noise artificially added for denoising task

3. Mathematical Formulation of Autoencoder

Autoencoder consists of:

Encoder: $z = f(x)$

Decoder: $x' = g(z)$

Objective:

Minimize reconstruction error:

Loss = $||x - x'||^2$

or Binary Cross Entropy

4. Algorithm Limitations

- Cannot generalize to unseen noise types
- May blur images
- Requires tuning of architecture
- Not suitable for very complex images

5. Methodology / Workflow

1. Load MNIST dataset
2. Normalize images
3. Add Gaussian noise
4. Build encoder-decoder architecture
5. Train model using noisy images as input and clean images as output
6. Predict denoised images
7. Visualize results

6. Performance Analysis

Observed Results:

Model successfully removes noise from images

Reconstructed images are close to original

Some smoothing occurs in output

Interpretation:

Autoencoder learns compressed representation of images

Decoder reconstructs clean version from noisy input

Model captures important visual features

7. Hyperparameter Configuration

Hidden layers: 128 → 64 → 128

Activation: ReLU and Sigmoid

Loss: Binary Crossentropy

Optimizer: Adam

Epochs: 10

Batch size: 256

OUTPUT:

Noisy images successfully denoised. Reconstructed images resemble original digits. Model demonstrates feature extraction capability

```
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape
```

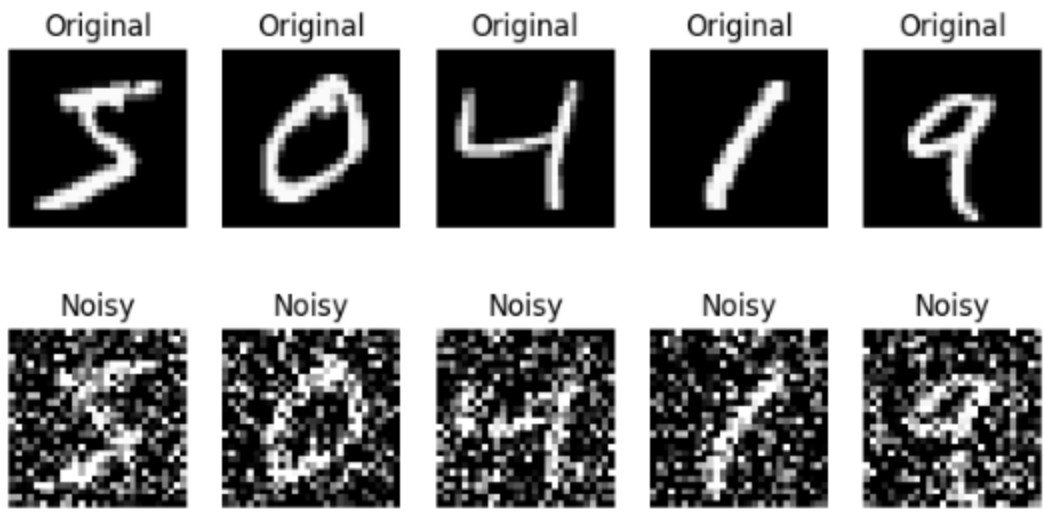
```
(X_train, _), (X_test, _) = mnist.load_data()

print(X_train.shape, X_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
(60000, 28, 28) (10000, 28, 28)
```

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
noise_factor = 0.5

X_train_noisy = X_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X_train.shape)
X_test_noisy = X_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X_test.shape)

X_train_noisy = np.clip(X_train_noisy, 0., 1.)
X_test_noisy = np.clip(X_test_noisy, 0., 1.)
```

```python
plt.figure(figsize=(8,4))

for i in range(5):
    plt.subplot(2,5,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title("Original")
    plt.axis('off')

    plt.subplot(2,5,i+6)
    plt.imshow(X_train_noisy[i], cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

plt.show()
```



```python
input_img = Input(shape=(28,28))

x = Flatten()(input_img)

# Encoder
x = Dense(128, activation='relu')(x)
x = Dense(64, activation='relu')(x)

# Decoder
x = Dense(128, activation='relu')(x)
x = Dense(784, activation='sigmoid')(x)

decoded = Reshape((28,28))(x)

autoencoder = Model(input_img, decoded)

autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

autoencoder.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28)	0
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 128)	8,320
dense_3 (Dense)	(None, 784)	101,136
reshape (Reshape)	(None, 28, 28)	0

Total params: 218,192 (852.31 KB)
Trainable params: 218,192 (852.31 KB)
Non-trainable params: 0 (0.00 B)

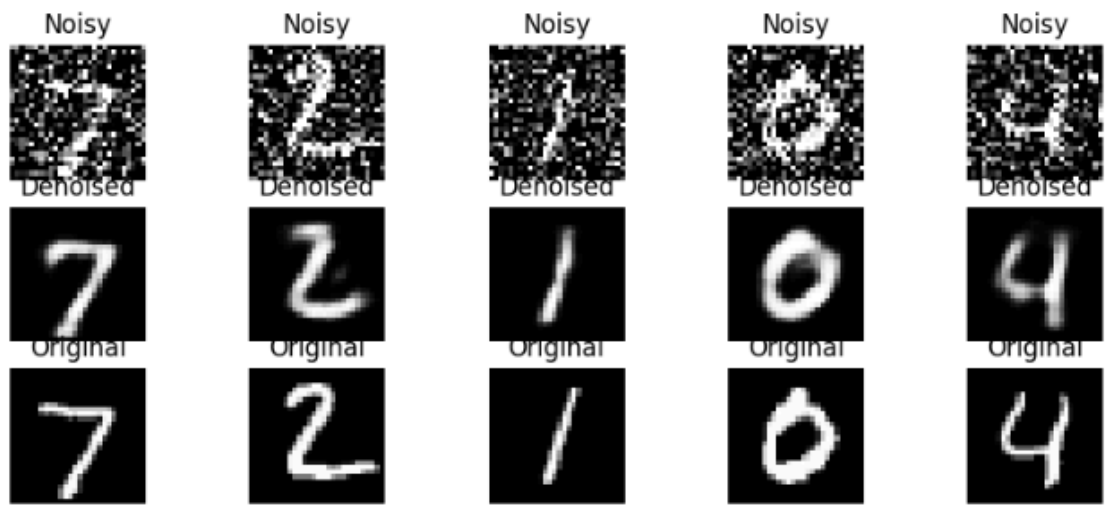
```python
history = autoencoder.fit(
    X_train_noisy, X_train,
    epochs=10,
    batch_size=256,
    validation_data=(X_test_noisy, X_test)
)
```

```
Epoch 1/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 20ms/step - loss: 0.2435 - val_loss: 0.1787
Epoch 2/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 24ms/step - loss: 0.1654 - val_loss: 0.1536
Epoch 3/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 19ms/step - loss: 0.1497 - val_loss: 0.1437
Epoch 4/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.1415 - val_loss: 0.1372
Epoch 5/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 25ms/step - loss: 0.1361 - val_loss: 0.1330
Epoch 6/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 10s 22ms/step - loss: 0.1320 - val_loss: 0.1297
Epoch 7/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 20ms/step - loss: 0.1291 - val_loss: 0.1271
Epoch 8/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 20ms/step - loss: 0.1269 - val_loss: 0.1250
Epoch 9/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 7s 29ms/step - loss: 0.1250 - val_loss: 0.1237
Epoch 10/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.1236 - val_loss: 0.1227
```

```
decoded_imgs = autoencoder.predict(X_test_noisy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
```

```
plt.figure(figsize=(10,4))

for i in range(5):
    # Noisy
    plt.subplot(3,5,i+1)
    plt.imshow(X_test_noisy[i], cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Denoised
    plt.subplot(3,5,i+6)
    plt.imshow(decoded_imgs[i], cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

    # Original
    plt.subplot(3,5,i+11)
    plt.imshow(X_test[i], cmap='gray')
    plt.title("Original")
    plt.axis('off')

plt.show()
```



CONCLUSION:

Autoencoders effectively remove noise from images, Model learns compressed representation of input data. Useful for image preprocessing and enhancement. Demonstrates unsupervised learning capability.Applicable in image restoration and compression.